

CS 551 Systems Programming

Lecture 5: Ownership and Borrowing

The rule Rust is actually famous for

Where we are

Lecture 4 answered *where* a value lives: the stack, the heap, and what a pointer carries. You can now draw the memory diagram.

It left the harder question untouched. Given a value somewhere in memory, and several parts of your program that want it, **who is allowed to read it, who is allowed to write it, and who frees it?**

C answers "you are, good luck." Garbage-collected languages answer "nobody has to think about it, and you pay at runtime." Rust answers with a rule the compiler checks. That rule is the borrow checker, and it is why the rest of this course is possible.

Today's path

1. **The ownership rule.** One value, one owner, and what a move is.
2. **Borrowing.** Access without ownership, and the rule that governs it.
3. **Lifetimes**, in one slide, as the thing that rule implies.
4. **Rc<T>**. What to do when one owner genuinely is not enough.

5. The borrow checker

Ownership, moves, and borrows

The ownership rule

Every value has exactly one owner at a time. When the owner goes out of scope, the value is dropped.

That's it. The entire system is built on that one rule.

- "Owner" is the variable binding holding the value.
- "Dropped" means the destructor runs and any owned resources (heap memory, file handles, sockets) are released.
- "At a time" is doing work. We'll see.

Move semantics

```
1 let s1 = String::from("hello");  
2 let s2 = s1;  
3  
4 println!("{}", s1);
```

Move semantics

```
1 let s1 = String::from("hello");
2 let s2 = s1;
3
4 println!("{}", s1);
```

The pointer, length, and capacity are copied. The bytes on the heap are **not**

Assignment in Rust *moves* the value, by default.

After `let s2 = s1`, `s1` is no longer a valid binding. There is still exactly one owner of the heap-allocated string: `s2`.

Contrast with C, where `s1` and `s2` would both hold the same pointer, giving you a double-free at end of scope. Contrast with Java, where `s1` and `s2` would alias and the GC would clean up later. Rust picks a third path: *transfer*, statically tracked.

Move semantics

```
1 let s1 = String::from("hello");  
2 let s2 = s1;  
3  
4 println!("{}", s1);
```

And **s1** is invalidated. It is no longer the owner, so it can do nothing with the data

Assignment in Rust *moves* the value, by default.

After `let s2 = s1`, `s1` is no longer a valid binding. There is still exactly one owner of the heap-allocated string: `s2`.

Contrast with C, where `s1` and `s2` would both hold the same pointer, giving you a double-free at end of scope. Contrast with Java, where `s1` and `s2` would alias and the GC would clean up later. Rust picks a third path: *transfer*, statically tracked.

Copy types are the exception

Small, fixed-size, no-resource types implement the `Copy` trait. For them, `=` *copies* instead of moving:

```
1 let a: i32 = 5;
2 let b = a;      // a is still valid; b is a fresh copy.
3 println!("{}", a, b); // fine.
```

Integers, floats, booleans, `char`, and tuples/arrays of those are `Copy`. `String`, `Vec`, `Box`, anything that owns a heap allocation, is **not** `Copy`.

Rule of thumb: if the type owns memory or another resource, it moves. If it's a "plain old number," it copies.

Borrowing: access without ownership

```
1 fn length(s: &String) -> usize {  
2     s.len()  
3 }  
4  
5 fn main() {  
6     let s = String::from("hello");  
7     let n = length(&s);  
8     println!("{}", s, n);  
9 }
```

Borrowing: access without ownership

```
1 fn length(s: &String) -> usize {  
2     s.len()  
3 }  
4  
5 fn main() {  
6     let s = String::from("hello");  
7     let n = length(&s);  
8     println!("{}", s, n);  
9 }
```

length takes a **reference** to a **String**, so it borrows and does not take ownership

Which is why **s** is still usable here.
The borrow ended when **length** returned

Because moving on every assignment would be exhausting, Rust lets you *borrow* a value instead. `length` borrows the string for the duration of the call. When it returns, the borrow ends and `s` is still the owner.

The borrow rules

At any point in your program, for any given value, *one* of these is true:

1. There are zero or more **shared references** (`&T`) to it. All read-only.
2. There is exactly **one mutable reference** (`&mut T`) to it. Read-write, exclusive.

That's the rule. **Many readers, or one writer, never both.**

This is the same invariant that makes data-race-free concurrency possible. It's also what prevents iterator invalidation, use-after-free, and a long list of other classic bugs. We'll see why in two slides.

LIVE CODING CHALLENGE

Take `let mut v = vec![1, 2, 3, 4, 5];`. Make a `&mut` slice of the first two elements and a `&mut` slice of the last three, then write through both.

The two slices do not overlap. Does it compile?

Pause here. Try it before the next slide.

The canonical break

```
1 fn main() {
2     let mut v = vec![1, 2, 3, 4, 5];
3
4     let a = &mut v[0..2];
5     let b = &mut v[2..5];    // <-- borrow checker rejects.
6
7     a[0] = 10;
8     b[0] = 20;
9 }
```

```
error[E0499]: cannot borrow `v` as mutable more than once at a time
```

`a` and `b` clearly point at *different* slots. But the compiler doesn't reason about indexes; it reasons about which value is borrowed. Since `v[0..2]` and `v[2..5]` both come from `v`, both borrows are against `v`, and the second one violates the rule.

The canonical break

```
1 fn main() {  
2     let mut v = vec![1, 2, 3, 4, 5];  
3  
4     let a = &mut v[0..2];  
5     let b = &mut v[2..5];    // <-- borrow checker rejects.  
6  
7     a[0] = 10;  
8     b[0] = 20;  
9 }
```

Reference rules. At any given time you can have one mutable reference *or* any number of immutable ones. References must always be valid.

```
error[E0499]: cannot borrow `v` as mutable more than once at a time
```

`a` and `b` clearly point at *different* slots. But the compiler doesn't reason about indexes; it reasons about which value is borrowed. Since `v[0..2]` and `v[2..5]` both come from `v`, both borrows are against `v`, and the second one violates the rule.

`split_at_mut`: when the compiler can't see it

For exactly this case, the standard library provides:

```
1 fn main() {
2     let mut v = vec![1, 2, 3, 4, 5];
3
4     let (a, b) = v.split_at_mut(2);    // returns two non-overlapping &mut [i32]
5     a[0] = 10;
6     b[0] = 20;
7
8     println!("{:?}", v);    // [10, 2, 20, 4, 5]
9 }
```

`split_at_mut` proves to the compiler (via `unsafe` code inside the standard library) that the two halves don't overlap.

Prove it once, inside `std`

The *user* of `split_at_mut` writes safe code. The proof is done once, by its author, and audited once.

This is the pattern for the whole language. Because the rule is deliberately conservative, escape hatches exist for the cases it rejects unfairly, and each one is audited once instead of per use.

Lifetimes, the one-slide version

Every reference has a **lifetime**: the region of code over which it's valid. Most of the time the compiler infers it.

```
1 fn first_word(s: &str) -> &str {  
2     // the returned reference lives as long as the input.  
3 }
```

The signature is sugar for `fn first_word<'a>(s: &'a str) -> &'a str`. The `'a` is a *lifetime parameter*; it says "the output reference is tied to the same lifetime as the input."

You'll write explicit lifetimes when the compiler can't infer them (multiple references in, one out, ambiguous which one feeds the return value). Until then, treat lifetimes as a thing the compiler is tracking for you.

What this prevents, concretely

The borrow rules look pedantic. What they buy you:

- **No use-after-free.** A reference can't outlive what it points to. The compiler enforces this.
- **No double-free.** Exactly one owner, exactly one drop. Move semantics, not aliasing.
- **No iterator invalidation.** You can't hold a `&mut` to a `Vec` and a `&` to its elements at the same time, so you can't push to a vector while iterating over it.
- **No data races.** Many readers or one writer is exactly the invariant a multithreaded program needs. (We'll cement this in the concurrency lecture.)

This is the payoff. The price is upfront design tax. The deal is good.

6. `Rc<T>`, when one owner isn't enough

Shared ownership

Sometimes you really do need two owners

The owner rule says one value, one owner. What if multiple parts of your program legitimately need to keep the same value alive?

Examples:

- A configuration object read by many subsystems.
- A node in a graph reachable from multiple paths.
- An immutable shared cache.

For these, you switch from a `Box` to an `Rc`.

Rc<T>: reference-counted ownership

`Rc<T>` is a smart pointer that **counts how many owners** a value has. The value lives on the heap; the count lives next to it. When the count drops to zero, the value is dropped.

```
1 use std::rc::Rc;
2
3 fn main() {
4     let a = Rc::new(String::from("hello"));           // count = 1
5     let b = Rc::clone(&a);                             // count = 2
6     let c = Rc::clone(&a);                             // count = 3
7
8     println!("count = {}", Rc::strong_count(&a));      // 3
9 } // <-- c, b, a drop in reverse order. count -> 2 -> 1 -> 0 -> freed.
```

`Rc::clone` is **not** a deep copy. It bumps the count and hands you another handle.

Caveat: `Rc` is single-threaded. For shared ownership *across threads*, you'd use `Arc`.

Box vs Rc, the decision

You want...	Reach for
One owner; heap allocation	<code>Box<T></code>
Many owners; one thread	<code>Rc<T></code>
Many owners; many threads	<code>Arc<T></code>
Many owners + interior mutation, one thread	<code>Rc<RefCell<T>></code>
Many owners + interior mutation, many threads	<code>Arc<Mutex<T>></code>

We've covered the top two today. The rest is the rest of the course.

Wrap-up

Recap

1. **Stack vs heap.** Default is stack; heap is opt-in through types like `Box`, `Vec`, `String`, `Rc`.
2. **References.** `&T` is shared and read-only; `&mut T` is exclusive and read-write.
3. **Fat pointers.** `&[T]` and `&str` are two words: pointer + length. Bounds-checking and UTF-8 safety ride on the metadata word.
4. `Box<T>`. Single-owner heap allocation. Use it for recursive types, trait objects, and large values.
5. **Ownership.** One owner; drop on scope exit. Move on assignment unless `Copy`.
6. **Borrow rules.** Many `&` or one `&mut`, never both. The whole memory-safety story rides on this.
7. `Rc<T>`. Reference-counted shared ownership when one owner isn't enough.

Exercises

1. Write a `cons_count(&List) -> usize` that returns the number of `Cons` cells in a list. Compare your version to `sum`. What's identical, what's different?
2. Build the broken two-`&mut` example yourself. Read the full error output. Fix it with `split_at_mut`. Print the result.
3. Predict, for each of these snippets, whether it compiles. Then run them.
 - `let v = vec![1, 2, 3]; let r = &v[0]; let s = &v[1]; println!("{}", r, s);`
 - `let mut v = vec![1, 2, 3]; let r = &v[0]; v.push(4); println!("{}", r);`
4. Replace one of your `Box<T>` cons-list examples with `Rc<List>`. What changes? When would you prefer one over the other?

Reading

- **The Rust Programming Language**, chapters 4 (Ownership), 8 (Common Collections, focus on `Vec`), and 15 (Smart Pointers).
- **Rust by Example**, sections on ownership and borrowing.
- **Optional, deeper.** Niko Matsakis's blog posts on the borrow checker; "Stacked Borrows" if you want to know how Miri models aliasing.
- **Course supplements.** [stack-and-heap-primer.md](#) if this was your first exposure to the stack and heap; [coming-from-c.md](#) if you came in with C or C++.

Next lecture: slices. Borrowing part of a collection, and the bug the borrow checker catches because of it.

Next: Lecture 6

Slices

A borrow that names part of something, and the bug class it deletes.